

AstraNotes

UML Design Package

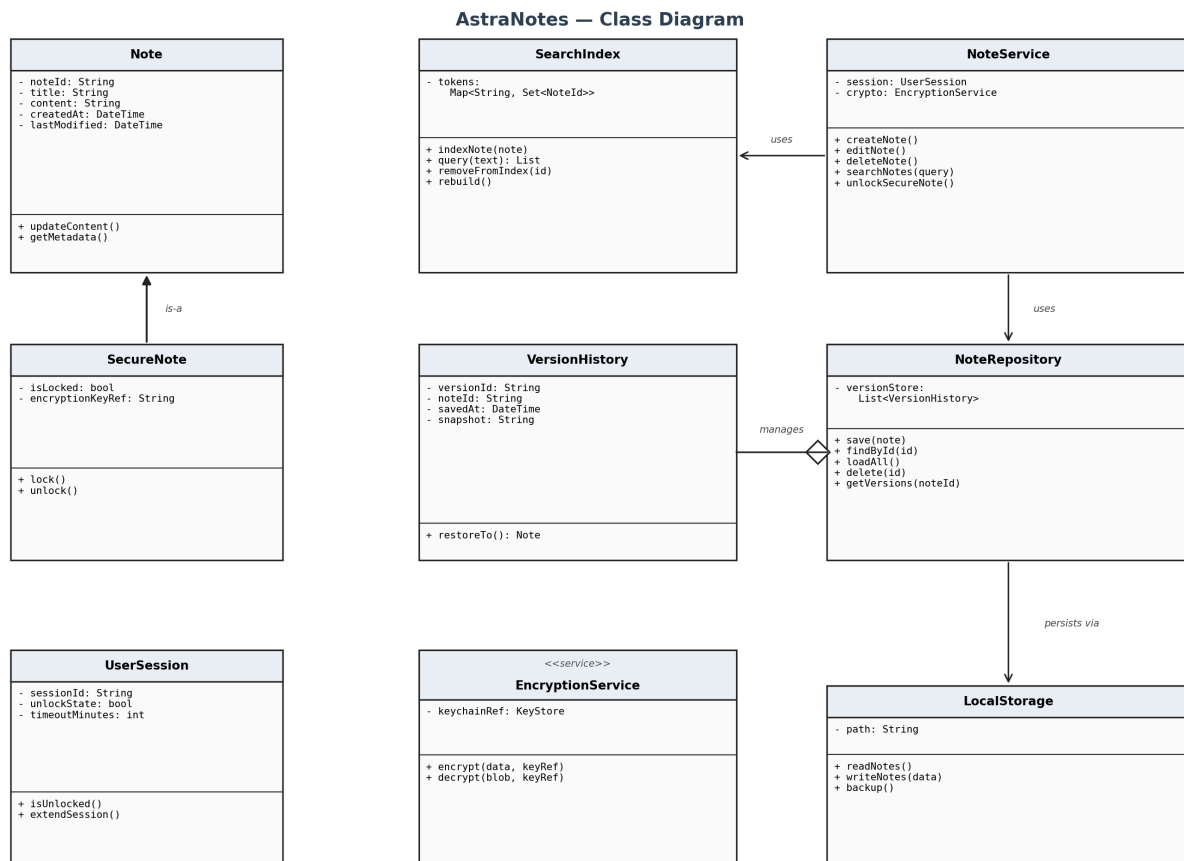
CSEN 296B-2 — AI-Driven Software Development

Week 4.2 Lab: Complete UML Design Package

Instructor: Paddu Melanahalli

Section	Contents
1	Class Diagram
2	Object Diagram
3	Use Case Diagram
4	Activity Diagram — Create & Save Secure Note
5	Deployment Diagram — Local-First Runtime
6	Rationale: How the Views Fit Together
7	Reflection on the Process

1. Class Diagram



The class diagram organizes AstraNotes into four layers. The **domain layer** (`Note`, `SecureNote`) holds note state; **SecureNote** extends **Note** through inheritance because a secure note genuinely is a kind of note that adds locking behavior and a reference to an encryption key. The **service layer** (`NoteService`) coordinates workflow and delegates to specialized services. The **persistence layer** (`NoteRepository`, `LocalStorage`) handles storage. **Cross-cutting services** (`EncryptionService`, `UserSession`) provide capabilities that the service layer composes.

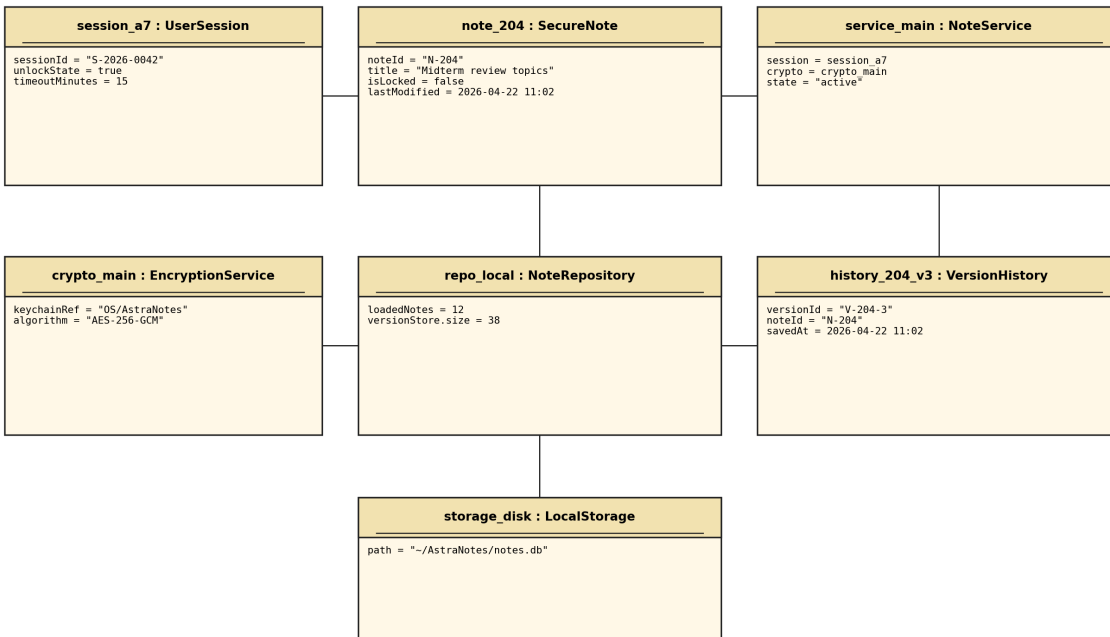
Three relationship types are used intentionally. *Inheritance* (`SecureNote` → `Note`) marks an is-a specialization. *Aggregation* (`NoteRepository` ■— `VersionHistory`) is shown with a hollow diamond rather than composition because version records need to outlive the notes they describe — if a user deletes a note, the audit trail and restore points should still be available through the repository. Modeling this as composition would have meant losing version history on delete, which contradicts the version-restoration use case.

Association with usage arrows covers the rest. `NoteService` uses `NoteRepository` and `SearchIndex`; `NoteRepository` persists through `LocalStorage`. `NoteService`'s dependence on `UserSession` and `EncryptionService` is conveyed through attribute typing (`session: UserSession`, `crypto: EncryptionService`) rather than additional dependency arrows, following standard UML practice that

attribute typing makes the dependency arrow redundant.

2. Object Diagram

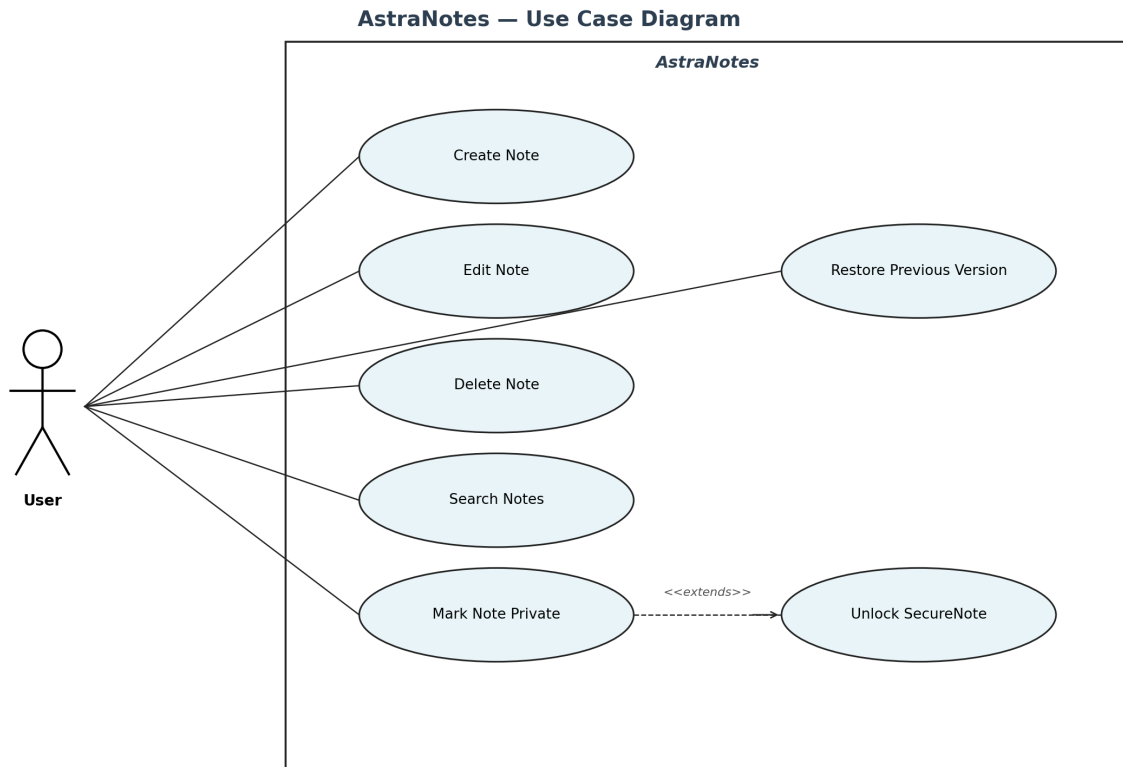
AstraNotes — Object Diagram (runtime snapshot)



The object diagram captures one runtime moment so the structural design does not stay abstract. In this snapshot, the user has an active **UserSession** (*session_a7*) with the secure note **N-204** currently unlocked. The **NoteService** instance (*service_main*) holds references to both the session and the **EncryptionService** (*crypto_main*) that handles AES-256-GCM operations. The note has just been edited, producing a new **VersionHistory** entry (*history_204_v3*) at 2026-04-22 11:02. The repository (*repo_local*) holds twelve loaded notes in memory and a version store of 38 history records — note that the version store size is larger than the loaded notes count because some history entries belong to notes that have since been deleted, which is exactly the behavior the aggregation relationship was chosen to support.

This view exists to answer a different question than the class diagram: the class diagram says *what can exist*; the object diagram says *what does exist right now*. It also shows that the static design produces a coherent runtime configuration — every reference between instances corresponds to a relationship in the class diagram.

3. Use Case Diagram

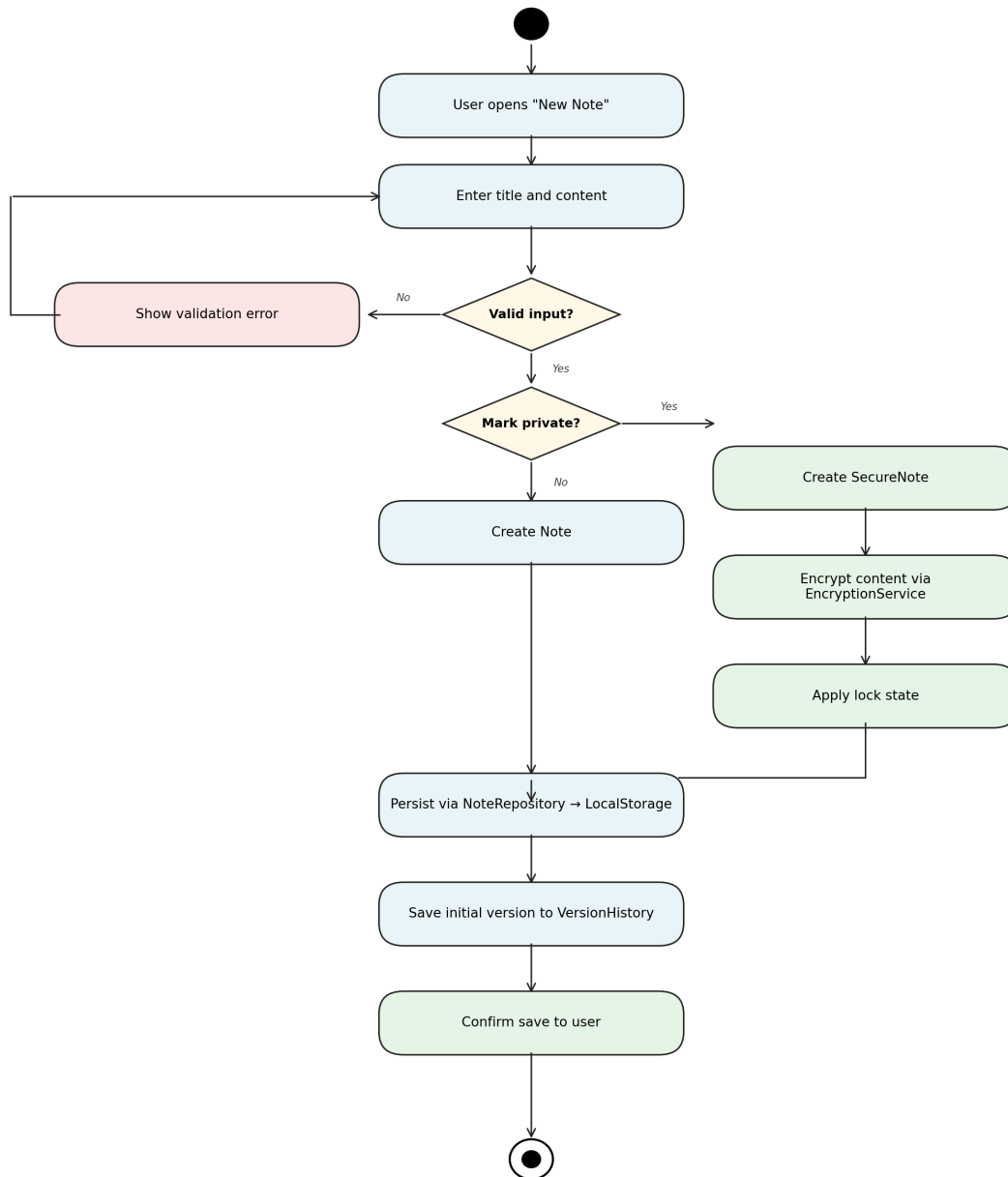


The use case diagram steps back from internal structure to focus on user goals. There is one primary actor — the **User** — and the system boundary contains the seven user-visible goals that justify everything inside the class diagram: create, edit, delete, and search notes; mark a note as private; unlock a secure note; and restore a previous version.

The **Unlock SecureNote** use case is connected to **Mark Note Private** through an **<<extends>>** relationship rather than a direct user association. This is intentional: the user does not start the day intending to *unlock* a note — they intend to read or edit one. The unlock step extends the main flow only when the targeted note is private and currently locked. **Restore Previous Version** is shown as a top-level user goal because version restoration is one of the things the aggregation choice on **VersionHistory** is meant to support — if the use case is real, the design has to support it.

4. Activity Diagram — Create & Save Secure Note

AstraNotes — Activity Diagram (Create & Save Secure Note)



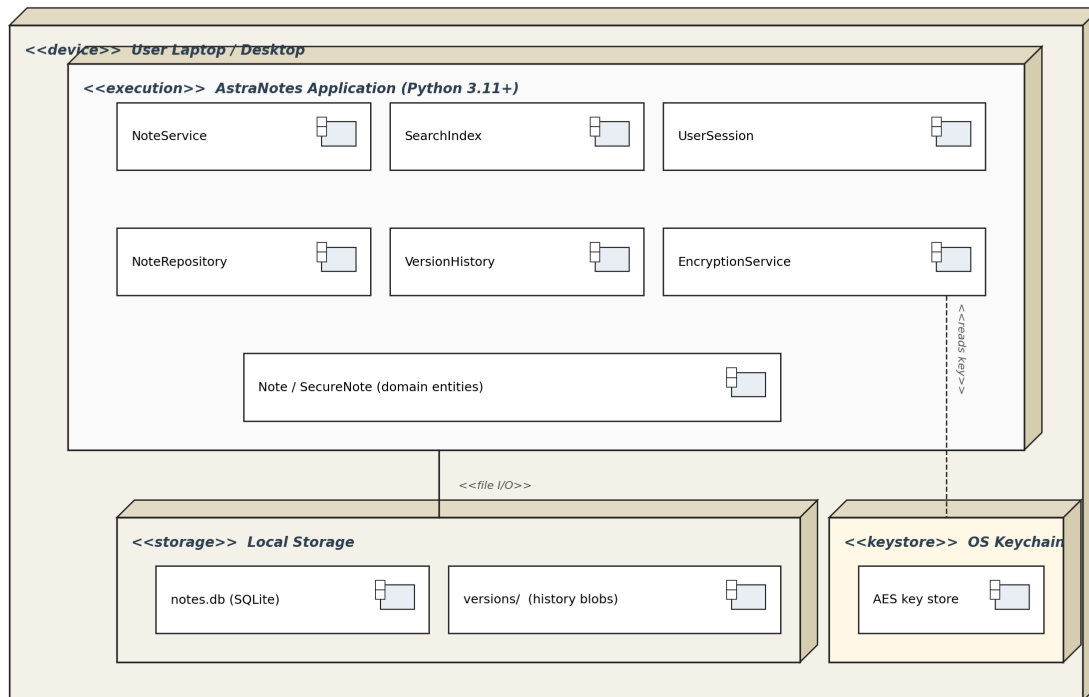
This activity diagram models a single workflow end-to-end: creating and saving a secure note. It was chosen rather than a more generic create-note flow because it exercises the parts of the design that carry the most risk — input validation, the privacy branch with encryption, persistence, and version capture — in one path.

Two decision points shape the flow. The first asks whether input is valid; if not, the user sees an error and the flow loops back to the input action so they can correct and retry, rather than terminating the

workflow. The second asks whether the note should be marked private; the Yes branch creates a **SecureNote**, encrypts its content through **EncryptionService**, and applies the lock state, while the No branch creates an ordinary **Note**. Both branches converge on persistence so every successful save leaves the system in a consistent state: written to local storage and captured in version history. The end node is reached only after the user is given an explicit save confirmation, which prevents silent failure.

5. Deployment Diagram — Local-First Runtime

AstraNotes — Deployment Diagram (local-first)



The deployment diagram shows where AstraNotes actually runs. Everything lives on the user's device. The outer node is the laptop or desktop; inside it is the AstraNotes execution environment running on Python 3.11+. Application components are organized in two rows that mirror the service / persistence layering of the class diagram: **NoteService**, **SearchIndex**, and **UserSession** on top; **NoteRepository**, **VersionHistory**, and **EncryptionService** below. Domain entities (**Note**, **SecureNote**) sit at the bottom of the application as the data layer. Persistence happens through file I/O to a local SQLite database (*notes.db*) and a versions directory holding history blobs.

The OS keychain is shown as a separate keystore node. **EncryptionService** depends on it via a dashed **<<reads key>>** relationship to retrieve AES keys for encrypt and decrypt operations. This separation is the most important security property in the design: encryption keys are never stored alongside note data. Even if *notes.db* were copied off the device, the secure-note contents would remain protected because the keys live in the OS keychain. The local-first posture directly supports the privacy concerns identified during the Week 3.2 governance and ethics review — there is no network boundary, no third-party service, and no telemetry path that note content could leak through.

6. Rationale: How the Views Fit Together

All five diagrams describe the same AstraNotes system, just from different angles. The class diagram answers *what exists*: domain entities, service responsibilities, and the relationships that hold across the system's lifetime. The object diagram grounds that structure with one runtime snapshot, which is the easiest way to confirm the static design produces a coherent moment of system state. The use case diagram steps back from internal structure to user goals, so the package stays honest about what the system is for.

The activity diagram zooms into one important workflow — creating and saving a secure note — because that path touches input validation, the privacy branch, encryption, persistence, and version capture in a single trace. The deployment diagram closes the loop by showing where the system actually runs and how the privacy boundary is enforced through the OS keychain rather than through application-level claims.

Consistency across views was treated as a hard constraint. Class names in the structural diagram appear unchanged in the object snapshot, the activity flow steps, and the deployment components. The privacy boundary is visible in three views at once: SecureNote and EncryptionService in the class diagram, the unlock-extends relationship in the use case diagram, and the keychain dependency in the deployment view. The activity flow's encryption step exists because EncryptionService exists; the version-save step exists because the aggregation between NoteRepository and VersionHistory exists. Each diagram supports claims the others make.

The package is also deliberately small. There is no cloud sync, no multi-user authentication, no enterprise deployment topology, because current AstraNotes requirements don't justify any of that. Inheritance is used only where SecureNote is genuinely a kind of Note. Aggregation rather than composition is used for VersionHistory because version records need to survive note deletion to support the restore-previous-version use case. Cross-cutting service dependencies (UserSession, EncryptionService) are expressed through attribute typing rather than additional arrows to keep the diagram readable.

Another engineer reading this should be able to recover the project's direction: a modular, local-first, privacy-aware note system where encryption is a first-class service rather than a leaked detail, version history is preserved across deletions, and the deployment posture matches the privacy claims made elsewhere in the project documentation.

7. Reflection on the Process

The hardest decisions in this package were not about which classes to draw but about which relationships to use between them. The first AI draft modeled VersionHistory as composition off Note, with a filled diamond. That looked clean on paper, but composition implies parts die with the whole — meaning if a user deletes a note, the version history for that note vanishes too. That contradicts the restore-previous-version use case, where the whole point is being able to recover state from before a deletion. I changed it to aggregation owned by NoteRepository, with a hollow diamond. Now versions outlive notes, which is what the design actually needs.

I also added EncryptionService as a separate class. The earlier draft had *encryptionKeyRef* as an attribute on SecureNote with no clear answer to where encrypt and decrypt operations actually happened. That's a leaked detail — security operations belong in a dedicated service, not smeared across a domain entity. The deployment diagram is also stronger because of this: there is now a visible boundary between data and keys, with the OS keychain as a separate keystore node, which makes the privacy claim concrete rather than aspirational.

Two arrows in the original draft were removed because they were noise. A NoteService → Note arrow labeled *manages* was redundant — every service manages the entities its method signatures already reference. A UserSession → SecureNote arrow conflated authentication with authorization; sessions don't gate notes directly, NoteService checks session state before performing operations on SecureNotes. Removing both arrows made the diagram easier to read and more accurate to how the code would actually be structured.

The activity diagram had a structural problem in the first version: the validation-error path went off the left edge and arrowed down to the end node, which read as "validation failed → terminate." That's not what should happen. I rerouted the loop back to the *Enter title and content* action so the user can correct input and retry, which is what the actual workflow needs to do.

The biggest takeaway from this lab is that AI is faster at producing UML than it is at justifying UML. The first draft had every reasonable class and a credible-looking layout, but several of the relationship choices didn't survive scrutiny. Composition vs aggregation, where encryption lives, what arrows are actually carrying information versus just adding noise — those are the kinds of architectural calls that need to be made deliberately. Most of the work on this lab was making those calls one at a time and then checking that the rest of the package still agreed with the new decision.